

Proficiency* Historian

USER API GUIDE

Version 6.0

June 2015



Disclaimer of Warranties and Liability

The information contained in this manual is believed to be accurate and reliable. However, GE Intelligent Platforms, Inc. assumes no responsibilities for any errors, omissions or inaccuracies whatsoever. Without limiting the foregoing, GE Intelligent Platforms, Inc. disclaims any and all warranties, expressed or implied, including the warranty of merchantability and fitness for a particular purpose, with respect to the information contained in this manual and the equipment or software described herein. The entire risk as to the quality and performance of such information, equipment and software, is upon the buyer or user. GE Intelligent Platforms, Inc. shall not be liable for any damages, including special or consequential damages, arising out of the use of such information, equipment and software, even if GE Intelligent Platforms, Inc. has been advised in advance of the possibility of such damages. The use of the information contained in the manual and the software described herein is subject to GE Intelligent Platforms, Inc. standard license agreement, which must be accepted by the buyer or user before the use of such information, equipment or software.

Trademark Notices

© 2015, General Electric Company. All rights reserved.

Proficiency is a trademark of GE Intelligent Platforms, Inc., a wholly-owned subsidiary of General Electric Company.

* Indicates a trademark of General Electric Company and/or its subsidiaries.

All other trademarks are the property of their respective owners.

All other product names and marks identified throughout this book are trademarks or registered trademarks of their respective companies. They are used throughout this book in editorial fashion only. No such use, or the use of any trade name, is intended to convey endorsement or affiliation.

No part of this publication may be reproduced in any form, or stored in a database or retrieval system, or transmitted or distributed in any form by any means, electronic, mechanical photocopying, recording or otherwise, without the prior written permission of GE Intelligent Platforms. Information contained herein is subject to change without notice.

We want to hear from you. If you have any comments, questions, or suggestions about our documentation, send them to the following email address:

doc@ge.com

Table of Contents

Proficy Historian User API	1
Overview of the Historian User API	1
Prerequisites	1
Functions	2
Connect Functions	2
Connect Functions	2
ihuConnect Function	2
ihuConnectEx Function	3
ihuDisconnect Function	4
ihuSetConnectionParameters Function	4
ihuRestoreDefaultConnectionParameters Function	5
ihuServerRegisterCallbacks Function	6
Interface Functions	7
ihuBrowseCollectors Function	7
Archiver Functions	7
Archiver Functions Overview	7
Archiver functions	7
ihuSetArchiverProperty Function	8
ihuGetArchiverProperty Function	11
Tag Functions	11
Tag Functions Overview	11
Tag Functions	12
Tag Property Values	13
ihuCreateTagCacheContext Function	17
ihuFetchTagCache Function	17
ihuFetchTagCacheEx Function	17

ihuFetchTagCacheEx2 Function	18
ihuFetchTagCacheEx3 Function	18
ihuGetTagNameCacheIndex Function	18
ihuGetTagNameCacheIndexEx2 Function	19
ihuGetNumericTagPropertyByTagname Function	19
ihuGetNumericTagPropertyByIndex Function	20
ihuGetNumericTagPropertyByIndexEx2 Function.....	20
ihuGetStringTagPropertyByTagName Function	21
ihuGetStringTagPropertyByTagName functionEx2.....	21
ihuGetStringTagPropertyByIndex Function	22
ihuGetStringTagPropertyByIndexEx2 Function.....	22
ihuTagAdd Function.....	23
ihuTagDelete Function	23
ihuTagDeleteEx Function	24
ihuTagRename Function	24
ihuTagRenameEx Function	25
ihuTagCacheCriteriaClear Function	25
ihuTagCacheCriteriaClearEx2 Function	25
ihuTagCacheCriteriaSetStringProperty Function	26
ihuTagCacheCriteriaSetStringPropertyEx2 Function	26
ihuTagCacheCriteriaSetNumericProperty Function	26
ihuTagCacheCriteriaSetNumericPropertyEx2 Function	27
ihuTagClearProperties Function.....	27
ihuTagSetStringProperty Function	27
ihuTagSetNumericProperty Function	28
ihuCloseTagCache Function	28
ihuCloseTagCacheEx2 Function	28
Write Functions.....	28

Write Functions Overview.....	28
Write Functions.....	29
ihuWriteData Function	29
ihuWriteComment Function	30
Query Modifiers Functions.....	31
Query Modifiers Functions Overview.....	31
Query Modifier Functions	31
ihuBrowseQueryModifiers Function.....	31
ihuClearQueryModifiers Function	32
ihuRetrieveCalculatedDataEx2 Function.....	32
ihuSetQueryModifiers Function	33
Read Functions.....	33
Read Functions Overview	33
Read Functions.....	34
ihuReadCurrentValue Function	34
ihuReadInterpolatedValue Function	34
ihuReadInterpolatedValueEx Function	36
ihuReadRawDataByTime Function	37
ihuReadRawDataByTimeEx Function	37
ihuReadMultiTagRawDataByTime Function	38
ihuReadMultiTagRawDataByTimeEx Function	38
ihuReadRawDataByCount Function.....	39
ihuReadRawDataByCountEx Function.....	39
ihuReadMultiTagRawDataByCount Function.....	40
ihuReadMultiTagRawDataByCountEx Function.....	41
ihuRetrieveSampledData Function.....	41
ihuRetrieveSampledDataEx Function.....	42
ihuRetrieveSampledDataEx2 Function.....	43

ihuRetrieveCalculatedData Function	43
ihuRetrieveCalculatedDataEx Function	44
ihuRetrieveCalculatedDataEx3 Function	44
Utility Functions	45
Utility Functions Overview	45
Utility Functions	45
IHU_timestamp_FromParts Function	45
IHU_timestamp_ToParts Function	46
EnumeratedSet Functions	47
Enumerated Sets Functions Overview	47
EnumeratedSet Functions	47
ihuGetEnumeratedSets Function	47
ihuEnumeratedSetAdd Function.....	48
ihuEnumeratedSetRawValue Function.....	48
ihuEnumeratedSetsFree Function.....	49
ihuEnumeratedSetRename Function	49
ihuEnumeratedSetDelete Function.....	50
ihuEnumeratedStateAdd Function.....	50
ihuEnumeratedStateModify Function	51
ihuEnumeratedStateDelete Function.....	52
UserDefinedType Functions	52
UserDefined Type Overview.....	52
User Defined Type Functions	52
ihuUserDefinedTypeAdd Function.....	53
ihuUserDefinedTypeDelete Function.....	53
ihuUserDefinedTypeRename Function	54
ihuUserDefinedTypeExists Function	54
ihuGetUserDefinedTypes Function	55

ihuUserDefinedTypeSetProperties Function	55
ihuUserDefinedTypeFreeProperties Function	56
Publish Functions	56
Publish Functions Overview	56
Publish Functions	57
ihuPublishAddTag Function.....	57
ihuPublishRemoveTag Function.....	58
ihuPublishTagCloseCache Function	58
ihuPublishGetTagPropertiesToCache Function	58
ihuPublishTagGetNumericPropertyByTagname Function.....	59
ihuPublishTagGetNumericPropertyByIndex Function	59
ihuPublishTagGetStringPropertyByTagname Function.....	60
ihuPublishTagGetStringPropertyByIndex Function	60
ihuPublishSetTagProperties Function	61
ihuPublishTagSetNumericProperty Function	61
ihuPublishTagSetStringProperty Function	62
ihuPublishTagClearProperties Function	62
ihuPublishGetDestinationServer Function.....	63
ihuPublishSetDestinationServer Function	63
Error Codes	64
Sample Programs	67
Sample Programs Overview.....	67
About compiling the samples.....	68
CollectorLike	68
PlotLike Sample Program	68
ReportLike Sample Program	68
MigrationLike Sample Program	68

Proficy Historian User API

Overview of the Historian User API

The Historian User API is intended to provide high speed read/write access to Historian data and read access to Historian tags. There is no access to archives, alarms, events, or messages.

Use the API to develop applications, which read and write data to the Historian server when the Historian SDK and Historian OLEDB do not meet your project requirements for performance or programming language.

Note: *At this time, the Historian User API supports single byte strings only.*

Sample programs are provided with the API, demonstrating how to perform common tasks. Additional information can be found in comments in the sample code and API header.

No guidance is given regarding application conversion from other products or APIs, including converting from the Historian SDK.

The reader is assumed to be familiar with the Historian features and functionality. Refer to the Historian Electronic Book for product information. This is especially important if you are using security groups with Historian, since the applications calling into the User API are limited by the security access granted at the server.

Prerequisites

Programs created with the Historian User API must be run on a machine with the following software installed and configured:

- Historian 6.0
- Historian 6.0 Client Tools

The Historian User API has been tested with Visual Studio .NET 2003, 2005, and 2008.

The Historian User API has no additional hardware requirements.

Proficy Historian allows you to develop both 32-bit and 64-bit User API programs.

NOTE: *If you want to build a 32-bit User API program on a 64-bit operating system, then you need to rename the `ihuapi32.lib` to `ihuapi.lib` and include it in your program.*

Functions

Connect Functions

This group of functions provides a means to connect to and disconnect from an Historian server. A minimal number of properties are exposed.

Connect Functions

- ihuConnect
- ihuConnectEx
- ihuDisconnect
- ihuSetConnectionParameters
- ihuRestoreDefaultConnectionParameters
- ihuServerRegisterCallbacks

ihuConnect Function

Use the ihuConnect function to connect to a Historian Server. The function will provide a server handle to be used in subsequent calls.

Prototype

```
ihuConnect { in MSO MSO Char * server, in MSO MSO Char * username, in MSO MSO Char * password, out long * serverhandle };
```

Remarks

The inputs to the function are server, username, and password. Each has a default value if NULL is passed.

Server - if NULL is passed, then the connection attempt will be to the local machine.

Username/password - if NULL is passed, then the username that owns the process is used. Most of the time this is the same as the user logged into the operating system. However, in the case of a program running as a service you can specify a username and password that the process should use.

The output of the function is a server handle.

Server handles are only valid for the lifetime of the process. They should not be saved to a file and reused.

You do not need to call connect more than one time for a username and password. If the connection to server was lost and restored, the handle can be used after reconnect. If the server was not there at

connect time, a handle will still be returned and you can use that as soon as the connection becomes available. Reconnects are performed inside of the API. The application should wait, retry reads and writes with the returned server handle. The reads and writes will succeed once the underlying connection is reestablished.

You should still call disconnect with the server handle returned, even if an error is returned.

A connection to the server will consume a Client Access License (CAL) only if you were not already accessing the server from this IP address. There is no way to connect without consuming a CAL.

Returns

ihuConnect returns the following values:

IhuSTATUS_OK

ihuSTATUS_FAILED

ihuSTATUS_API_TIMEOUT

ihuSTATUS_NOT_VALID_USER

ihuSTATUS_LIC_TOO_MANY_USERS

ihuConnectEx Function

Use the ihuConnectEx function to connect to a server with the support of store and forward.

Prototype

```
ihuErrorCode IHUAPI ihuConnectEx ( MSO Char * server, MSO Char * username,
MSO Char * password, MSO Char * buffername, unsigned long MaxMegMemory,
unsigned long MinMegDiskFree, long * serverhandle );
```

Remarks

The inputs to the function are server, username, password, buffername, MaxMegMemory, and MinMegDiskFree.

Server - if NULL is passed, then the connection attempt will be to the local machine.

Username/password - if NULL is passed, then the username that owns the process is used. Most of the time this is the same as the user logged into the operating system. However, in the case of a program running as a service you can specify a username and password that the process should use.

Buffer Name - The filename and location to store buffered data. Make sure the buffer file name is unique.

MaxMegMemory - Maximum memory in MB. Buffered data is stored in this memory until it is full, later it is stored to disk.

MinMegDiskFree - Minimum free disk space in MB.

The output of the function is a server handle.

Server handles are only valid for the lifetime of the process. They should not be saved to a file and reused.

You do not need to call connect more than one time for a username and password. If the connection to server was lost and restored, the handle can be used after reconnect. If the server was not there at connect time, a handle will still be returned and you can use that as soon as the connection becomes available. Reconnects are performed inside of the API. The application should wait, retrying reads and writes with the returned server handle. The reads and writes will succeed once the underlying connection is reestablished.

You should still call disconnect with the server handle returned, even if an error is returned.

A connection to the server will consume a Client Access License (CAL) only if you were not already accessing the server from this IP address. There is no way to connect without consuming a CAL.

Returns

ihuConnect returns the following values:

IhuSTATUS_OK

IhuSTATUS_FAILED

IhuSTATUS_API_TIMEOUT

IhuSTATUS_NOT_VALID_USER

IhuSTATUS_LIC_TOO_MANY_USERS

ihuDisconnect Function

Use the ihuDisconenct function to release connection resources.

Prototype

```
ihuDisconnect { in long serverhandle };
```

Returns

ihuDisconnect returns the following values:

IhuSTATUS_OK

ihuSetConnectionParameters Function

Use the ihuSetConnectionParameters function to set the socket connection timeout.

Prototype

```
ihuSetConnectionParameters { in IHU_CONNECTION_PARAMETERS*Params };
```

Remarks

The input to the function is IHU_CONNECTION_PARAMETERS structure to pass in a connection timeout

```
/* Client-side, global connection parameters */

typedef struct {

    int Size; // Structure size in bytes

    int TCPConnectionWindow; // Max time to establish a TCP connection with server
    in seconds (default 5s)

} IHU_CONNECTION_PARAMETERS;
```

The default connection timeout is 5 seconds. There is no maximum value, but it is not recommended to set the value to more than 60 seconds. If you increase the timeout, attempts to connect a server will take longer time to return if the server is not available.

Sample Code

If an archiver is busy to process the connections, then you can set a longer timeout using the following code:

```
IHU_CONNECTION_PARAMETERS params;

params.Size = sizeof(IHU_CONNECTION_PARAMETERS);

params.TCPConnectionWindow = 30; // extend window to 30s

ihuSetConnectionParameters(&params);
```

NOTE: This code only applies to the connections made from your program. You need to make this call each time you run your program.

Returns

ihuSetConnectionParameters returns the following values:

ihuSTATUS_OK on success

ihuSTATUS_FAILED on fail

ihuRestoreDefaultConnectionParameters Function

Use the ihuRestoreDefaultConnectionParameters function to reset all connection parameters such as the socket connection timeout to default values.

Prototype

```
ihuRestoreDefaultConnectionParameters { void, };
```

Remarks

Resets the connection timeout to 5 seconds

Returns

IhuSTATUS_OK

ihuServerRegisterCallbacks Function

Use the ihuServerRegisterCallbacks function if you want your program to be notified of changes in buffering or connection state. For example, you can be notified of connection loss or buffering becoming full.

Prototype

```
ihuErrorCode IHUAPI ihuServerRegisterCallbacks ( long hServer, void
*UserParameter, long *RegisterCallbacksStatus, void *BufferCallbackFunction,
void *ConnectionCallbackFunction );
```

Remarks

The inputs to the function are hServer, UserParameter, RegisterCallbacksStatus, BufferCallbackFunction, and ConnectionCallbackFunction.

hServer - server handle from the connect. The callbacks are as per server handle, so you can have callbacks for some connections and not others or you can have different callbacks for different handles.

UserParameter - NULL or an integer value that you want passed back to you in the callback.

RegisterCallbacksStatus - The success or failure of setting up the callbacks..

BufferCallbackFunction - callback function for buffer state changes.

ConnectionCallbackFunction - callback function for connection state changes.

Returns

ihuServerRegisterCallbacks returns the following values:

IhuSTATUS_OK

IhuSTATUS_FAILED

Interface Functions

ihuBrowseCollectors Function

Use the ihuBrowseCollectors function to browse collectors that are connected to the archiver.

Prototype

```
ihuErrorCode IHUAPI ihuBrowseCollectors ( long hServer, MSO Char  
*InterfaceNameMask, IHU_COLLECTOR **Collectors, int *NumOfCollectors );
```

Remarks

The inputs to the function are hServer and InterfaceNameMask.

hServer - server handle indicating which server to be browsed.

***InterfaceNameMask** - pass * for all interfaces, or interface name or detailed mask for efficiency.

The output of the function is.

****Collectors** - returns list of interfaces / collectors.

***NumOfCollectors** - returns number of interfaces found.

Returns

ihuBrowseCollectors returns the following values:

IhuSTATUS_OK

IhuSTATUS_FAILED

Archiver Functions

Archiver Functions Overview

This group of functions provides a means to read and write a set of archiver properties. A minimum of properties (only those related to reads, writes, and tag browses) are available. See the code comments in the Historian User API header, *ihuapi.h* for more details

Archiver functions

- ihuGetArchiverProperty
- ihuSetArchiverProperty

ihuSetArchiverProperty Function

Use the ihuSetArchiveProperty function to change the value of specific Historian server options.

Prototype

```
ihuSetArchiverProperty
(
    long serverhandle,
    MSO Char *ArchiveProperty,
    MSO Char *PropertyValue)

```

Remarks

The following properties are supported:

Property	Read/Write Access	Values	Description
ARCHIVER_PROP_CREATEOFFLINEARCHIVES	Read/Write	1, 0	This property is normally set to "0" (FALSE) implies that writes before the start time of the earliest archive are not allowed. Set to "1" only if you need to migrate legacy data and then put it back to "0" when you are done migrating. Leaving this set to "1" can cause excessive archive creation, especially when data is sent to the archiver out of order.

Property	Read/Write Access	Values	Description
ARCHIVER_PROP_CONFIGSERIALNUMBER	Read only		This property is a large number that changes when the tag configuration changes. The typical use of this is to cache configuration properties like the set of tags. You can tell when your cache becomes out of date by comparing your saved serial number with the current serial number.
ARCHIVER_PROP_ACTIVEHOURS	Read/Write	1-232800 hours (30 years)	This property indicates how many hours prior to now, that the archive data becomes read only. If you are trying to migrate legacy data then you may need to set this to a large value.
SECURITY_PROP_STRICTCLIENTAUTHENTICATION	Read/Write	1, 0	This property indicates whether strict client authentication is enabled or not. By default, it is set to "1" (TRUE) on new Historian installations. If it is set to TRUE, only version 4.7 or newer clients are given access to the Historian server.

Property	Read/Write Access	Values	Description
SECURITY_PROP_STRICTCOLLECTORAUTHENTICATION	Read/Write	1, 0	This property indicates whether strict collector authentication is enabled or not. By default, it is set to "1" (TRUE) on new Historian installations. If it is set to TRUE, only version 4.7 or newer collectors are given access to the Historian server.

Returns

ihuSetArchiverProperty returns the following values:

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_OK

ihuSTATUS_ACCESS_DENIED

To change this property...	You need to be a member of this group...
ARCHIVER_PROP_CREATEOFFLINE ARCHIVES	ih Archive Admins or ih Security Admins
ARCHIVER_PROP_ACTIVEHOURS	ihSecurity Admins
SECURITY_PROP_STRICTCLIENTAUTHENTICATION	ihSecurity Admins

To change this property...	You need to be a member of this group...
SECURITY_PROP_STRICTCOLLECT ORAUTHENTICATION	ihSecurity Admins

ihuGetArchiverProperty Function

Use the ihuGetArchiverProperty function to read the value of certain archiver properties.

Prototype

```
ihuGetArchiveProperty { in long serverhandle, in MSO Char *ArchiveProperty,
out MSO Char *PropertyValue };
```

Remarks

The user need only be a member of ih Readers security group to read the properties. The user does not need to be in ih Security Admins or ih Archive Admins unless they want to change property values.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_ACCESS_DENIED

Tag Functions

Tag Functions Overview

The group of tag functions provides browse/add/modify/delete access to the configured tags on a server. For each tag, the tag properties can be retrieved with User API calls. An example of this use would be to retrieve user defined collection parameters for a tag prior to collection. Or, user configured display parameters can be retrieved from a tag in preparation for plotting.

Both string and numeric tag properties can be retrieved. Every tag property is classified as string or numeric (double float). For example, Boolean properties are converted to double float and time data type properties are converted to string.

You can add, delete, or modify tags using User API.

You can browse tags in 3 different ways depending if you need complex criteria or if you need multiple browses open at the same time. The simplest browse uses ihuFetchTagCache which lets you filter by tagname. If you need to filter by additional properties such as data type or collectorname, use

the `ihuFetchTagCacheEx`. Finally if you need to have multiple browses open at the same time, such as browsing all the tags on one server and another server and comparing them, use the `ihuFetchTagCacheEx2` or `ihuFetchTagCacheEx3`.

Tag Functions

- `ihuFetchTagCache`
- `ihuFetchTagCacheEx`
- `ihuFetchTagCacheEx2`
- `ihuFetchTagCacheEx3`
- `ihuGetTagNameCacheIndex`
- `ihuGetTagNameCacheIndexEx2`
- `ihuGetNumericTagPropertyByTagname`
- `ihuGetNumericTagPropertyByIndex`
- `ihuGetNumericTagPropertyByIndexEx2`
- `ihuGetStringTagPropertyByTagname`
- `ihuGetStringTagPropertyByTagnameEx2`
- `ihuGetStringTagPropertyByIndex`
- `ihuGetStringTagPropertyByIndexEx2`
- `ihuCloseTagCache`
- `ihuTagAdd`
- `ihuTagDelete`
- `ihuTagRename`
- `ihuTagCacheCriteriaClear`
- `ihuTagCacheCriteriaClearEx2`
- `ihuTagCacheCriteriaSetStringProperty`
- `ihuTagCacheCriteriaSetStringPropertyEx2`
- `ihuTagCacheCriteriaSetNumericProperty`
- `ihuTagCacheCriteriaSetNumericPropertyEx2`
- `ihuTagClearProperties`
- `ihuTagSetStringProperty`
- `ihuTagSetNumericProperty`
- `ihuCloseTagCache`
- `ihuCloseTagCacheEx2`

Tag Property Values

Overview

This is the current set of tag properties exposed, and an indication if they are numeric or string. Use the `ihuGetNumericTagPropertyByIndex` and `ihuGetNumericTagPropertyByTagname` functions for numeric properties and the `ihuGetStringTagPropertyByIndex` and `ihuGetStringTagPropertyByTagname` functions to retrieve string properties.

This Tag Property...	Has this type of value...
<code>ihuTagPropTagname</code>	String
<code>ihuTagPropDescription</code>	String
<code>ihuTagPropEngineeringUnits</code>	String
<code>ihuTagPropComment</code>	String
<code>ihuTagPropDataType</code>	Numeric
<code>ihuTagPropFixedStringLength</code>	Numeric
<code>ihuTagPropInterfaceName</code>	String
<code>ihuTagPropSourceAddress</code>	String
<code>ihuTagPropCollectionType</code>	Numeric
<code>ihuTagPropCollectionInterval</code>	Numeric

This Tag Property...	Has this type of value...
ihuTagPropCollectionOffset	Numeric
ihuTagPropLoadBalancing	Numeric
ihuTagPropTime StampType	Numeric
ihuTagPropHighEngineeringUnits	Numeric
ihuTagPropLowEngineeringUnits	Numeric
ihuTagPropInputScaling	Numeric
ihuTagPropHighScale	Numeric
ihuTagPropLowScale	Numeric
ihuTagPropCollectorCompression	Numeric
ihuTagPropCollectorDeadbandPercentRange	Numeric
ihuTagPropArchiveCompression	Numeric
ihuTagPropArchiveDeadbandPercentRange	Numeric

This Tag Property...	Has this type of value...
ihuTagPropSpare1	String
ihuTagPropSpare2	String
ihuTagPropSpare3	String
ihuTagPropSpare4	String
ihuTagPropSpare5	String
ihuTagPropReadSecurityGroup	String
ihuTagPropWriteSecurityGroup	String
ihuTagPropAdministratorSecurityGroup	String
ihuTagPropLastModified not currently implemented	String
ihuTagPropLastModifiedUser	String
ihuTagPropInterfaceType	Numeric
ihuTagPropStoreMilliseconds	Numeric

This Tag Property...	Has this type of value...
ihuTagPropUTCBias	Numeric
ihuTagPropNumberOfCalculationDependencies	Numeric
ihuTagPropCalculationDependencies only the first dependency is returned	String
ihuTagPropAverageCollectionTime	Numeric
ihuTagPropCollectionDisabled	Numeric
ihuTagPropArchiveCompressionTimeout	Numeric
ihuTagPropCollectorCompressionTimeout	Numeric
ihuTagPropDataDensity	Numeric <i>NOTE: The tag Data Density is of the following categories Minimum (1), Medium (4), Maximum (7).</i>
ihuTagPropNumberOfElements	Numeric <i>NOTE: If the NumberOfElements value is -1 then the tag is an array tag.</i>

ihuCreateTagCacheContext Function

This function creates tag cache context to be used in the ihuFetchTagCacheEx2 or ihuFetchTagCacheEx3..

Prototype

```
void * IHUAPI ihuCreateTagCacheContext { };
```

ihuFetchTagCache Function

This function fetches a current set of tags and properties from the Historian server and places them into an API cache for subsequent queries, such as ihuGetNumericTagPropertyByTagname.

Prototype

```
ihuFetchTagCache { in long serverhandle, in MSO Char * tagmask, out int * NumTagsFound };
```

Remarks

The function retrieves properties for each tag received.

There is a single API tag cache. The previous tag cache must be freed before doing another fetch from the same or a different server. Keep this in mind when accessing the tag cache from multiple threads in the same application. Or, use the ihuCreateTagCacheContext and ihuFetchTagCacheEx2 or ihuFetchTagCacheEx3 functions.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuFetchTagCacheEx Function

This function fetches a current set of tags and properties from the Historian server and places them into an API cache for subsequent queries, such as ihuGetNumericTagPropertyByTagname.

Prototype

```
ihuFetchTagCacheEx { in long serverhandle, out int * NumTagsFound, };
```

Remarks

The function uses the criteria setup by ihuTagCacheCriteriaSetStringProperty and ihuTagCacheCriteriaSetNumericProperty for tag browse.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuFetchTagCacheEx2 Function

This function fetches a current set of tags and properties from the Historian server and places them into an API cache for subsequent queries.

Prototype

```
ihuFetchTagCacheEx2 { void * tagCacheContext, long serverhandle, MSO Char *  
tagmask, int * NumTagsFound };
```

Remarks

The function uses the criteria setup by ihuTagCacheCriteriaSetStringProperty and ihuTagCacheCriteriaSetNumericProperty for tag browse.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuFetchTagCacheEx3 Function

This function fetches a current set of tags and properties from the Historian server and places them into an API cache for subsequent queries.

Prototype

```
ihuFetchTagCacheEx3 { void * tagCacheContext, long serverhandle, int *  
NumTagsFound };
```

Remarks

The function uses the criteria setup by ihuTagCacheCriteriaSetStringProperty and ihuTagCacheCriteriaSetNumericProperty for tag browse.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuGetTagNameCacheIndex Function

This function finds the index of the given tagname in the cache. The index can be used for fast access in subsequent get property by index calls. This call only queries the API tag cache. It does not access the archiver.

Prototype

```
ihuGetTagnameCacheIndex { in MSO Char *Tagname out unsigned int *CacheIndex
};
```

Remarks

The index returned in CacheIndex is a zero based index suitable to be passed into the get property by index functions. The index is only valid if the function returns a success error code.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_TAGNAME

ihuGetTagnameCacheIndexEx2 Function

This function finds the index of the given tagname in the cache at thread level context. The index can be used for fast access in subsequent get property by index calls. This call only queries the API tag cache. It does not access the archiver.

Prototype

```
ihuGetTagnameCacheIndexEx2 { Void *TagCacheContext, MSO Char *Tagname
unsigned int *CacheIndex };
```

Remarks

The index returned in CacheIndex is a zero based index suitable to be passed into the get property by index functions. The index is only valid if the function returns a success error code.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_TAGNAME

ihuGetNumericTagPropertyByTagname Function

This function retrieves the value of a given numeric tag property from the API tag cache for a given tag. This call does not query the archiver, it queries the cache. The tag property may have changed since the cache was fetched.

Prototype

```
ihuGetNumericTagPropertyByTagname { in MSO Char *Tagname, in
ihuTagProperties TagProperty, out double *Value };
```

Remarks

The get by tagname function is slower than the get by index function because it has to loop through all

tags in the cache looking for the requested tag. If you will be retrieving multiple properties for a tag, consider calling the `ihuGetTagnameCacheIndex` function to get the cache index of a tag so that the get by index functions can be used.

Returns

`ihuSTATUS_OK`

`ihuSTATUS_INVALID_PARAMETER`

`ihuSTATUS_INVALID_TAGNAME`

ihuGetNumericTagPropertyByIndex Function

This function retrieves the value of a given numeric tag property from the API tag cache at a given cache index. This call does not query the archiver, it queries the cache. The tag property may have changed since the cache was fetched.

Prototype

```
ihuGetNumericTagPropertyByIndex { in int Index, in ihuTagProperties  
TagProperty, out double *Value };
```

Remarks

Use the get property by index functions when you have already located the tag using the `ihuGetTagnameCacheIndex` or when you want to iterate through all tags in the cache.

The index is zero based.

Tags are not returned from the cache in any particular order.

Returns

`ihuSTATUS_OK`

`ihuSTATUS_INVALID_PARAMETER`

`ihuSTATUS_INVALID_TAGNAME`

ihuGetNumericTagPropertyByIndexEx2 Function

This function retrieves the value of a given numeric tag property from the API tag cache at a given cache index. This call does not query the archiver, it queries the cache. The tag property may have changed since the cache was fetched.

Prototype

```
ihuGetNumericTagPropertyByIndexEx2 { void * TagCacheContext, int Index,  
ihuTagProperties TagProperty, double *Value };
```

Remarks

Use the get property by index functions when you do not know the tagname.

The index is zero based.

Tags are not returned from the cache in any particular order.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_INVALID_TAGNAME

ihuGetStringTagPropertyByTagName Function

This function retrieves the value of a given string tag property from the API tag cache for a given tag. This call does not query the archiver, it queries the API cache. The tag property may have changed since the cache was fetched.

Prototype

```
ihuGetStringTagPropertyByTagName { in MSO Char *Tagname, in
ihuTagProperties TagProperty, in int valuelength, out MSO Char *Value };
```

Remarks

The get by tagname function is slower than the get by index function because it has to loop through all tags in the cache looking for the requested tag. If you will be retrieving multiple properties for a tag, consider calling the ihuGetTagNameCacheIndex function to get the cache index of a tag so that the get by index functions can be used.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_INVALID_TAGNAME

ihuGetStringTagPropertyByTagName functionEx2

This function retrieves the value of a given string tag property from the API tag cache for a given tag. This call does not query the archiver, it queries the API cache. The tag property may have changed since the cache was fetched.

Prototype

```
ihuGetStringTagPropertyByTagNameEx2 { void * TagCacheContext, MSO Char *
Tagname, ihuTagProperties TagProperty, MSO Char * Value, int valuelength, };
```

Remarks

The get by tagname function is slower than the get by index function because it has to loop through all tags in the cache looking for the requested tag. If you will be retrieving multiple properties for a tag, consider calling the ihuGetTagNameCacheIndex function to get the cache index of a tag so that the get

by index functions can be used.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_INVALID_TAGNAME

ihuGetStringTagPropertyByIndex Function

This function retrieves the value of a given string tag property from the API tag cache at a given cache index. This call does not query the archiver, it queries the cache. The tag property may have changed since the cache was fetched.

Prototype

```
ihuGetStringTagPropertyByIndex { in int Index, in ihuTagProperties  
TagProperty, out MSO Char *Value, in int valuelength, };
```

Remarks

Use the get property by index functions when you have already located the tag using the ihuGetTagNameCacheIndex or when you want to iterate through all tags in the cache.

The index is zero based.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_INVALID_TAGNAME

ihuGetStringTagPropertyByIndexEx2 Function

This function retrieves the value of a given string tag property from the API tag cache at a given cache index. This call does not query the archiver, it queries the cache. The tag property may have changed since the cache was fetched.

Prototype

```
ihuGetStringTagPropertyByIndexEx2 { void * TagCacheContext, int Index,  
ihuTagProperties TagProperty, MSO Char * Value, int valueLength, };
```

Remarks

Use the get property by index functions when you do not the tag name or when you want to iterate through all tags in the cache.

The index is zero based.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_INVALID_TAGNAME

ihuTagAdd Function

Use this function to add or modify tags.

Prototype

```
ihuTagAdd( in long serverhandle, );
```

Remarks

This function uses serverhandle as a parameter.

If the tag name contains special characters "?" and "*", then the tag will not be added to the server.

To modify a tag, you must add a tag again with the new set of properties. Whenever you change or copy a tag name, the information about the old tag name, new tag name, and time stamps are all recorded in the audit trail.

Returns

ihuSTATUS_OK on success.

ihuSTATUS_INVALID_PARAMETER on invalid input parameters.

ihuSTATUS_LIC_TOO_MANY_TAGS if the tag add would exceed the licensed tag count.

ihuSTATUS_ACCESS_DENIED if you do not have permission to add tags.

ihuTagDelete Function

This function deletes a tag from the server.

Prototype

```
ihuTagDelete ( in long serverhandle, in MSO Char * tagname, );
```

Remarks

This function uses serverhandle and tagname as parameters.

Returns

ihuSTATUS_OK on success

ihuSTATUS_INVALID_PARAMETER on invalid input parameters

ihuSTATUS_ACCESS_DENIED if you do not have permission to add tags.

ihuTagDeleteEx Function

You can use this function to permanently delete a tag by passing TRUE as a parameter.

Prototype

```
ihuTagDelete { in long serverhandle, in MSO Char * tagname, BOOL  
DeletePermanent };
```

Remarks

Use permanent delete if you want to create a new tag with the previous name, The ihuTagDeleteEx uses the following as parameters.

- **serverhandle:** Parameter used to fetch from a specified Historian server.
- **tagname:** Name of the tag which you want to delete. You can only use this to delete tags that are returned in a tag browse.
- **DeletePermanent:** Permanently deletes a tag if the value is TRUE.

***NOTE:** If you permanently delete a tag you can no longer query the data of that tag, and the tag name is available for re-use.*

Returns

ihuSTATUS_OK on success

ihuSTATUS_INVALID_PARAMETER on invalid input parameters.

ihuSTATUS_ACCESS_DENIED if you do not have permission to delete tags. You must be a member of ihTagAdmin security group to be able to delete tags.

ihuTagRename Function

Use this function to rename tag names.

Prototype

```
ihuTagRename{ in long serverhandle, in MSO Char * OldTagName, in MSO Char  
* NewTagName, };
```

Remarks

This function uses serverhandle, OldTagName, and NewTagName as parameters.

Server Handle - Parameter used to fetch from a specified Historian 4.0 server.

OldTagname - Name of the old tag for which want to rename.

NewTagname: - Name of the new tag.

Tag renaming will only update or modify tag names without modifying or changing the tag properties. If you want to modify a renamed tag property, be aware that of all the alias' tag properties will also be

updated

Returns

ihuSTATUS_OK on success

ihuSTATUS_INVALID_PARAMETER on invalid input parameters

ihuSTATUS_ACCESS_DENIED if you do not have permission to add tags.

ihuTagRenameEx Function

You can use this function to permanently rename a tag by passing TRUE as a parameter.

Prototype

```
ihuTagRenameEx{ in long serverhandle, in MSO Char * OldTagName, in MSO
Char * NewTagName,
    BOOL TrueRename, };
```

Remarks

You can do a permanent rename if you do not want to read and write a tag with its previous name. The ihuTagRenameEx function uses the following parameters.

- **serverhandle:** Parameter used to fetch from a specified Historian server.
- **OldTagName:** Name of the old tag for which you want to rename.
- **NewTagName:** Name of the new tag.
- **TrueRename:** Permanently renames a tag if the value is TRUE.

Returns

ihuSTATUS_OK on success

ihuSTATUS_INVALID_PARAMETER on invalid input parameters

ihuSTATUS_ACCESS_DENIED if you do not have permission to add tags. You must be a member of ihTagAdmin security group to be able to rename tags.

ihuTagCacheCriteriaClear Function

Use this function to clear the criteria before setting up the tag browse criteria.

Prototype

```
void IHUAPI ihuTagCacheCriteriaClear(); }
```

ihuTagCacheCriteriaClearEx2 Function

Use this function to clear the criteria before setting up the tag browse criteria.

Prototype

```
void IHUAPI ihuTagCacheCriteriaClearEx2( void * TagCacheContext, );
```

ihuTagCacheCriteriaSetStringProperty Function

Use this function to setup the tag browse criteria before calling ihuFetchTagCacheEx.

Prototype

```
ihuTagCacheCriteriaSetStringProperty { in ihuTagProperties TagProperty, in  
MSO Char *Value, };
```

Remarks

ihuFetchTagCacheEx fetches a current set of tags and properties from the Historian server and places them into an API cache for subsequent queries, such as ihuGetNumericTagPropertyByTagname.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_INVALID_TAGNAME

ihuTagCacheCriteriaSetStringPropertyEx2 Function

Use this function to setup the tag browse criteria before calling ihuFetchTagCacheEx.

Prototype

```
ihuTagCacheCriteriaSetStringPropertyEx2 { void * TagCacheContext,  
ihuTagProperties TagProperty, MSO Char *Value, };
```

Remarks

ihuFetchTagCacheEx fetches a current set of tags and properties from the Historian server and places them into an API cache for subsequent queries, such as ihuGetNumericTagPropertyByTagname.

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_INVALID_TAGNAME

ihuTagCacheCriteriaSetNumericProperty Function

Use this function to setup the tag browse criteria.

Prototype

```
ihuTagCacheCriteriaSetNumericProperty { in ihuTagProperties TagProperty, in double Value, };
```

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_INVALID_TAGNAME

ihuTagCacheCriteriaSetNumericPropertyEx2 Function

Use this function to setup the tag browse criteria.

Prototype

```
ihuTagCacheCriteriaSetNumericPropertyEx2 { void * TagCacheContexty, ihuTagProperties TagProperty, double Value, };
```

Returns

ihuSTATUS_OK

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_INVALID_TAGNAME

ihuTagClearProperties Function

Use this function to clear the tag properties before adding tags in to a server.

Prototype

```
void IHUAPI ihuTagClearProperties() ;;
```

ihuTagSetStringProperty Function

Use this function to set a string tag property before calling ihuTagAdd.

Prototype

```
ihuTagSetStringProperty{ in ihuTagProperties TagProperty, in MS0 Char *Value, };
```

Returns

ihuSTATUS_OK

ihuTagSetNumericProperty Function

Use this function to set a numeric tag property before calling ihuTagAdd .

Prototype

```
ihuTagSetNumericProperty( in ihuTagProperties TagProperty, in double.Value,  
    );
```

Returns

ihuSTATUS_OK

ihuCloseTagCache Function

This function will free the API cache from memory. You should always close the tag cache when not in use to free memory and prevent accidental usage.

Prototype

```
ihuCloseTagCache { };
```

Returns

ihuSTATUS_OK always

ihuCloseTagCacheEx2 Function

This function will free the API tag cache context from memory. You should always close the tag cache when not in use to free memory and prevent accidental usage.

Prototype

```
ihuCloseTagCacheEx2 { void * TagCacheContext };
```

Returns

ihuSTATUS_OK always

Write Functions

Write Functions Overview

The write functions are designed for high performance data write access to the archiver. Groups of data samples are built up in the application and then sent as a whole into the API for transmission to the server. Any write errors can be returned to the application, which can then implement its own error handling. Or, the program can write without a wait if there is no error handling.

The application is responsible for all error handling, including retrying failed writes.

Write Functions

ihuWriteData

ihuWriteComment

ihuWriteData Function

This is a high performance function for writing multiple samples for multiple tags

Prototype

```
ihuWriteData { in long serverhandle, in int number_of_samples, in
IHU_DATA_SAMPLE *data_values, in ihuErrorCode *error_returns , in bool
wait_for_reply, in bool error_on_replace };
```

Important: You cannot write more than 100,000 samples at in one call to this function. It is recommended that you write 1000 to 10,000 samples per call.

Remarks

The Historian archiver has some strict rules regarding data writes, which apply to all applications and collectors:

1. No application is allowed to write data with a time stamp older than now minus the "Data is read only after" active hours setting. Such writes will fail.
2. No application is allowed to write data with a time stamp before the start time of the first archive, even if the write satisfies the previous rule.
3. No application can write data to a read only archive.
4. No application can write data with a time stamp more than 15 minutes in future.

The write function has two Boolean parameters to control optional behavior:

wait_for_reply - Setting this to TRUE will cause the write to block until the values have been acknowledged by the archiver. This has the benefit of allowing you to check the error returns to reach a degree of confidence that the data was written successfully. You cannot check error codes unless you wait for the reply.

If it is important that control be returned to your program so that collection can be performed, consider setting wait_for_reply = FALSE. Another reason to set wait_for_reply = FALSE would be if you have no specific error handling strategy or you are verifying write success by doing data readbacks.

If you are using store and forward in the User API, set wait_for_reply=FALSE

NOTE: If you set wait_for_reply = FALSE, then you must pass NULL error array for "ihuErrorCode *error_returns" value.

error_on_replace - This has no effect on performance but is used to tell the archiver how to react when an existing archived sample has the same time stamp as one being written. Setting error_on_replace = TRUE means don't overwrite, discard the new sample, and return an error which you can see if you have wait_for_reply = TRUE. Setting error_on_replace = FALSE means overwrite the archived sample with the new sample.

Time stamps:

The function call can specify a time stamp or pass a time stamp of zero seconds, telling the User API to use the current PC time on the written sample. Pass a time stamp structure where the seconds = 0. Do not pass NULL. All samples in the group that have a zero time stamp will get the same time stamp, which is the PC time at the time of the write. This time stamp has microseconds.

Values:

Values to be written do not have to be in the same data type as the Historian tag. The archiver will convert the written data type to the tag data type if necessary.

Qualities:

The function call is required to specify the data quality and subquality for the sample. There are no default values.

Note: Though the IHU_DATA_SAMPLE has a field for comments you cannot write a comment using this function. Use the ihuWriteComment function.

Error handling

The User API provides no store and forward or retries of failed writes. That is the responsibility of the application. A user API program can perform its own retries or use store and forward.

If you get a timeout back from a write, the write may still happen. The write may be waiting in a write queue on the Data Archiver.

Security

Historian has audited and unaudited writers security groups and a security administration group. The username performing the write must be a member of one of these groups or you will get an ihuSTATUS_ACCESS_DENIED error when you attempt to write.

Returns

The function returns ihuSTATUS_OK when the values were successfully written.

The function can return errors on timeouts or when user is not a member of the necessary security group or if the tag was not found.

Important: The application is responsible for accounting for the time differences between client and archiver if they are on different machines. The User API will pass through any timestamps given to it, without adjusting for any time offsets.

ihuWriteComment Function

This function writes a single comment to a single data sample.

Prototype

```
ihuWriteComment { in long serverhandle, in MSO Char *Tagname, in  
IHU_timestamp *time stamp, in MSO Char *Comment, in MSO Char *SuppliedUser,
```

```
in MSO Char *SuppliedPassword };
```

Remarks

Comments can be written with any time stamp that is valid for a data write. See the `ihuWriteData` function description for details on what a valid time stamp is. If there is a raw sample already for that tag and time stamp, then the comment is attached to that without losing the data value stored with the existing sample. Otherwise a new raw sample is created with no value to hold the comment.

If a time stamp with zero seconds is passed in, the current time will be used.

Currently, comments written with the User API can only be string text.

The `SuppliedUser` and `SuppliedPassword` are optional and can be NULL. If provided, they must be a valid username and password or the comment add will fail. The supplied username and password, in addition to being a valid OS user account, must also have write permissions or the write will fail.

Returns

`ihuSTATUS_OK` on success, error otherwise.

Query Modifiers Functions

Query Modifiers Functions Overview

The group of Query Modifier functions can be used to specify various ways of retrieving data from Proficy Historian. For example, user can request only good quality data with the modifier `ONLYGOOD` and/or retrieve replaced values with `INCLUDEREPLACED` modifier.

The following are the Query Modifier functions:

Query Modifier Functions

- `ihuBrowseQuery`
- `ihuClearQueryModifiers`
- `ihuRetrieveCalculatedDataEx2`
- `ihuSetQueryModifiers`

ihuBrowseQueryModifiers Function

This function returns the list of all supported query modifiers from the Proficy Historian Server. Different versions of the Historian Data Archiver may support different set of modifiers.

Prototype

```
ihuBrowseQueryModifiers( long serverhandle,          IHU_QUERY_MODIFIER
                        **QueryModifiers, int *NumberOfModifiers );
```

Remarks

The NumberOfModifiers returns the count of modifiers. QueryModifiers returns the list of supported query modifiers.

Returns

ihuSTATUS_OK on success

ihuSTATUS_INVALID_PARAMETER on invalid input parameters

ihuClearQueryModifiers Function

This function clears any query modifiers' that were previously set so that they will not be used in subsequent reads.

Prototype

```
ihuErrorCode ihuClearQueryModifiers(void)
```

Remarks

Use ihuSetQueryModifiers() to set a modifier string to be used in all reads. Call ihuClearQueryModifiers() to stop using that modifier.

Returns

ihuSTATUS_OK or ihuSTATUS_OUT_OF_MEMORY

ihuRetrieveCalculatedDataEx2 Function

This function returns calculated data based on the raw samples stored in archive. Use this function for ihuStateCount and ihuStateTime calculation modes.

Data can be requested using a number of samples or an interval. Set one to non-zero and the other to zero. To split the duration, divide the time from start to finish into evenly spaced time intervals.

Prototype

```
ihuRetrieveCalculatedDataEx2 (long serverhandle, IHU_TIMESTAMP StartTime,  
IHU_TIMESTAMP EndTime, ihuCalculationMode CalculationMode, ihuDataType  
StateDataType, ihuValue StateValue, unsigned long NumberOfSamples,  
IHU_DATA_INTERVAL Interval, IHU_RETRIEVED_DATA_RECORDS_EX *DataRecords)
```

Remarks

None.

Returns

The function returns ihuSTATUS_OK if the values are retrieved successfully.

The function can return errors on:

- Read timeouts
- If the user is not a member of the iH Readers security group
- If the tag was not found.

ihuSetQueryModifiers Function

This function sets up the query modifiers criteria to be used by all subsequent data reads.

Prototype

```
ihuSetQueryModifiers(  
    long serverhandle,  
    MSO Char *CriteriaString  
)
```

Remarks

None.

Returns

ihuSTATUS_OK on success

ihuSTATUS_INVALID_PARAMETER

ihuSTATUS_OUT_OF_MEMORY

Read Functions

Read Functions Overview

The read functions exported from the User API give applications the ability to retrieve raw, sampled, and calculated values stored in the Historian server. Retrieved samples can be used in reporting, plotting, or data analysis applications.

In most cases, multiple tags can be read in a single function call.

Most of the read functions serve the general purpose of retrieving raw, sampled or calculated data. Additionally, targeted functions are exposed, which provide easy access to a tag's current value or a tag's value interpolated to a given date and time stamp.

Comments, if they exist, will be returned with the data. See the sample programs for more information.

Read functions can be combined with write functions to perform read back of data just written. Read back is the most secure way to verify that the data was written successfully.

Read Functions

- `ihuReadCurrentValue`
- `ihuReadInterpolatedValue`
- `ihuReadRawDataByTime`
- `ihuReadMultiTagRawDataByTime`
- `ihuReadRawDataByCount`
- `ihuReadMultiTagRawDataByCount`
- `ihuRetrieveSampledData`
- `ihuRetrieveCalculatedData`
- `ihuReadFilterToggleData`

`ihuReadCurrentValue` Function

This function returns the current value of the tag. The current value has a specific definition in Historian but it is basically the last raw sample sent to the archiver for that tag.

Prototype

```
ihuReadCurrentValue { in long serverhandle, in int number_of_tags,  
in/out IHU_DATA_SAMPLE *pSamples, out ihuErrorCode *error_returns };
```

Remarks

You can retrieve the current value of multiple tags with one call. A single sample for each tagname requested will be returned.

For each tag requested, fill in only the tagname field in the `IHU_DATA_SAMPLE`. All other fields are set by the function.

The data will be returned based on the data type of the tag.

Returns

The function returns errors in two ways. The function has a return code and each tagname has an error code. Check the function level error return before examining the tag level error.

Both are `ihuSTATUS_OK` when the current value was successfully retrieved.

The function can return errors on read timeouts or when user is not a member of the `ih Readers` security group.

Each tag can return an error in scenarios such as when tag not found or when tag has no raw samples.

`ihuReadInterpolatedValue` Function

This function returns the value of a tag interpolated to a given date and time.

Prototype

```
ihuReadInterpolatedValue { in long serverhandle, in int number_of_tags,
in/out IHU_DATA_SAMPLE * pSamples, out ihuErrorCode * error_returns };
```

Remarks

You can retrieve the value of multiple tags with one call. A single sample for each tagname requested will be returned.

For each tag requested, fill in only the tagname and time stamp field in the IHU_DATA_SAMPLE. All other fields are set by the function. The time stamp in the first requested tag will be the one used.

The data will be returned based on the data type of the tag.

The data quality returned will be either ihuOPCGood or ihuOPCBad and the substatus will always be ihuOPCNonspecific.

To specify a time stamp, you must add it to the IHU_DATA_SAMPLE of the first tag. The following is an example:

// you only need to populate the time stamp in the first sample

```
lRet = IHU_timestamp_FromParts(2003,
7,
22,
11,
0,
0,
0,
&(pSamples[0].time stamp));

lRet = ihuReadInterpolatedValue (serverhandle,      // handle from connect
number_of_tags,      // number of tags, one sample per tag
pSamples,            // allocated by caller, value set by API
error_returns);      // error code per tag
```

An example can be found in the ReportLike sample program.

Returns

The function returns errors in two ways. The function has a return code and each tagname has an error

code. Check the function level error return before examining the tag level error.

Both are `ihuSTATUS_OK` when the value was successfully retrieved.

The function can return errors on read timeouts or when user is not a member of the `ih Readers` security group.

Each tag can return an error in scenarios such as when tag not found or when tag has no raw samples.

ihuReadInterpolatedValueEx Function

This function returns the number of samples of a tag interpolated to a given date and time with filters and query modifiers using `pszFilterExpression` and `CriteriaString`.

Prototype

```
ihuErrorCode IHUAPI ihuReadInterpolatedValueEx ( long serverhandle, // [in]
used for communication with server int number_of_tags, // [in] the number of
tags to retrieve IHU_DATA_SAMPLE *pSamples, // [in/out] user fills tagname
and timestamp and the API fills other fields MSO_Char *pszFilterExpression,
// [in] Filter Expression e.g. Tag > 20 IHU_FILTER_MODE FilterMode, //
[in] Filter Modes MSO_Char *CriteriaString, // [in] QueryModifiers to use
with data reads ihuErrorCode *error_returns // [out] populated with per tag
error );
```

Remarks

If you do not want to do a filtering you can pass `NULL` value to the `pszFilterExpression` and `ihuExactTime` to the `FilterMode` or you can just use the `ihuReadInterpolatedValue()` function.

Pass `NULL` value to the `CriteriaString` if you do not have a `QueryModifier`.

You can retrieve the value of multiple tags with one call. A single sample for each tagname requested will be returned.

For each tag requested, fill in only the tagname and time stamp field in the `IHU_DATA_SAMPLE`. All other fields are set by the function. The time stamp in the first requested tag will be the one used.

The data will be returned based on the data type of the tag.

The data quality returned will be either `ihuOPCGood` or `ihuOPCBad` and the substatus will always be `ihuOPCNonspecific`.

To specify a timestamp, you must add it to the `IHU_DATA_SAMPLE` of the first tag.

Returns

The function returns errors in two ways. The function has a return code and each tagname has an error code. Check the function level error return before examining the tag level error.

Both are `ihuSTATUS_OK` when the value was successfully retrieved.

The function can return errors on:

- Read timeouts.
- If the user is not a member of the ih Readers security group.
- If the tag was not found.
- If the filter criteria or query modifier cannot be set.

ihuReadRawDataByTime Function

This function returns multiple raw samples for a single tag in a given time range

Prototype

```
ihuReadRawDataByTime { in long serverhandle, in MSO_Char * tagname, in
IHU_timestamp * StartTime, in IHU_timestamp * EndTime, out int *
number_of_samples, out IHU_DATA_SAMPLE **data_values };
```

Remarks

The time stamp, value, and quality of each raw sample are returned.

Returns

The function returns ihuSTATUS_OK when the values were successfully retrieved.

The function can return errors on read timeouts or when user is not a member of the ih Readers security group or if the tag was not found.

ihuReadRawDataByTimeEx Function

This function returns multiple raw samples for a single tag in a given time range with filters and query modifiers using pszFilterExpression and CriteriaString.

Prototype

```
ihuReadRawDataByTimeEx { long serverhandle, // [in] the serverhandle MSO_Char
*tagname, // [in] the single tagname to fetch data for MSO_Char
*pszFilterExpression, // [in] Filter Expression e.g. Tag > 20 IHU_FILTER_MODE
FilterMode, // [in] Filter Modes MSO_Char *CriteriaString, // [in]
QueryModifiers to use with data reads IHU_TIMESTAMP * StartTime, // [in]
Start time of query IHU_TIMESTAMP * EndTime, // [in] End Time of query int *
number_of_samples, // [out] the number of samples returned IHU_DATA_SAMPLE
**data_values // [out] the returned data samples, unlimited number };
```

Remarks

If you do not want to do a filtering you can pass NULL value to the pszFilterExpression and ihuExactTime to the FilterMode or you can just use the ihuReadRawDataByTime() function.

Pass NULL value to the CriteriaString if you do not have a QueryModifier.

The timestamp, value, and quality of each raw sample are returned.

Returns

The function returns `ihuSTATUS_OK` when the values were successfully retrieved.

The function can return errors on:

- Read timeouts
- If the user is not a member of the iH Readers security group
- If the tag was not found.
- If the filter criteria or query modifier cannot be set.

ihuReadMultiTagRawDataByTime Function

This function returns multiple raw samples for a multiple tags in a given time range

Prototype

```
ihuReadMultiTagRawDataByTime { in long serverhandle, in int numeroftags,  
in IHU_timestamp * StartTime, in IHU_timestamp * EndTime, out ihuErrorCode  
**error returns, in/out IHU_RETRIEVED_RAW_VALUES *pSamples };
```

Remarks

The time stamp, value, and quality of each raw sample are returned.

Returns

The function returns `ihuSTATUS_OK` when the values were successfully retrieved.

The function can return errors on read timeouts or when user is not a member of the iH Readers security group or if the tag was not found.

ihuReadMultiTagRawDataByTimeEx Function

This function returns multiple raw samples for multiple tags in a given time range with filters and query modifiers using `pszFilterExpression` and `CriteriaString`.

Prototype

```
ihuReadMultiTagRawDataByTimeEx { long serverhandle, // [in] the serverhandle  
int number_of_tags, // [in] the number of tags to retrieve MSO_Char  
*pszFilterExpression, // [in] Filter Expression e.g. Tag > 20 IHU_FILTER_MODE  
FilterMode, // [in] Filter Modes MSO_Char *CriteriaString, // [in]  
QueryModifiers to use with data reads IHU_TIMESTAMP * StartTime, // [in]  
Start time of query IHU_TIMESTAMP * EndTime, // [in] End Time of query  
ihuErrorCode **error_returns, // [out] populated with per tag error  
IHU_RETRIEVED_RAW_VALUES *pSamples // [in/out] the returned data samples,  
unlimited number memory is allocated by the IHUAPI and must be freed by the  
caller by calling ihuFreePtr. };
```

Remarks

If you do not want to do a filtering you can pass NULL value to the pszFilterExpression and ihuExactTime to the FilterMode or you can just use the ihuReadMultiTagRawDataByTime() function.

Pass NULL value to the CriteriaString if you do not have a QueryModifier.

The timestamp, value, and quality of each raw sample are returned.

Returns

The function returns ihuSTATUS_OK when the values were successfully retrieved.

The function can return errors on:

- Read timeouts
- If the user is not a member of the iH Readers security group
- If the tag was not found.
- If the filter criteria or query modifier cannot be set.

ihuReadRawDataByCount Function

This function returns a given number raw samples for a single tag starting at a given time stamp and going forward or backward in time. If there are fewer samples than specified in the number_of_samples parameter, the function will stop returning data.

Prototype

```
ihuReadRawDataByCount { in long serverhandle, in MSO Char *tagname, in
IHU_timestamp * StartTime, in/out int * number_of_samples, in int
TimeForward, out IHU_DATA_SAMPLE **data_values };
```

Remarks

The time stamp, value, and quality of each raw sample are returned.

Returns

The function returns ihuSTATUS_OK when the values were successfully retrieved.

The function can return errors on read timeouts or when user is not a member of the ih Readers security group or if the tag was not found.

ihuReadRawDataByCountEx Function

This function returns a given number raw samples for a single tag starting at a given time stamp and going forward or backward in time with filters and query modifiers using pszFilterExpression and CriteriaString. If there are fewer samples than specified in the number_of_samples parameter, the function will stop returning data.

Prototype

```
ihuReadRawDataByCountEx { long serverhandle, // [in] the serverhandle
MSO_Char *tagname, // [in] the single tagname to fetch data for MSO_Char
*pszFilterExpression, // [in] Filter Expression e.g. Tag > 20 IHU_FILTER_MODE
FilterMode, // [in] Filter Modes MSO_Char *CriteriaString, // [in]
QueryModifiers to use with data reads IHU_TIMESTAMP * StartTime, // [in]
Start time of query

IHU_TIMESTAMP * EndTime, // [in] End time of query int * number_of_samples,
// [in/out] the number of samples to return and actually returned int
TimeForward, // [in] TRUE if search should be in forward time order
IHU_DATA_SAMPLE **data_values // [out] the returned data samples, unlimited
number };
```

Remarks

If you do not want to do a filtering you can pass NULL value to the pszFilterExpression and ihuExactTime to the FilterMode or you can just use the ihuReadRawDataByCount() function.

Pass NULL value to the CriteriaString if you do not have a QueryModifier.

The timestamp, value, and quality of each raw sample are returned.

Returns

The function returns ihuSTATUS_OK when the values were successfully retrieved.

The function can return errors on:

- Read timeouts.
- If the user is not a member of the iH Readers security group.
- If the tag was not found.
- If the filter criteria or query modifier cannot be set.

ihuReadMultiTagRawDataByCount Function

This function returns upto the user requested number of raw samples for a given set of tags.

Prototype

```
ihuErrorCode IHUAPI ihuReadMultiTagRawDataByCount ( long serverhandle, int
number_of_tags, IHU_TIMESTAMP *StartTime, int *number_of_samples, BOOL
TimeForward, ihuErrorCode **error_returns, IHU_RETRIEVED_RAW_VALUES *pSamples
);
```

Remarks

The function returns a per tag error and overall error.

Returns

The function returns ihuSTATUS_OK when the values were successfully retrieved.

The function can return errors on read timeouts or when user is not a member of the ih Readers security group or if the tag was not found.

ihuReadMultiTagRawDataByCountEx Function

This function returns the number of raw samples the user has requested for a given set of tags with filters and query modifiers using pszFilterExpression and CriteriaString.

Prototype

```
ihuErrorCode IHUAPI ihuReadMultiTagRawDataByCountEx ( long serverhandle, //
[in] the serverhandle int number_of_tags, // [in] the number of tags to
retrieve IHU_TIMESTAMP * StartTime, // [in] Start time of query

IHU_TIMESTAMP * EndTime, // [in] End time of query MSO_Char
*pszFilterExpression, // [in] Filter Expression e.g. Tag > 20 MSO_Char
*CriteriaString, // [in] QueryModifiers to use with data reads
IHU_FILTER_MODE FilterMode, // [in] Filter Modes int * number_of_samples,
// [in/out] the number of samples to return and actually returned BOOL
TimeForward, // [in] TRUE if search should be in forward time order
ihuErrorCode ** error_returns, // [out] populated with per tag error
IHU_RETRIEVED_RAW_VALUES *pSamples // [in/out] the returned data samples,
unlimited number memory is allocated by the IHUAPI and must be freed by the
caller by calling ihuFreePtr );
```

Remarks

If you do not want to do a filtering you can pass NULL value to the pszFilterExpression and ihuExactTime to the FilterMode or you can just use the ihuReadMultiTagRawDataByCount() function.

Pass NULL value to the CriteriaString if you do not have a QueryModifier.

The function returns a per tag error and an overall error.

Returns

The function returns ihuSTATUS_OK when the values were successfully retrieved.

The function can return errors on:

- Read timeouts
- If the user is not a member of the iH Readers security group
- If the tag was not found.
- If the filter criteria or query modifier cannot be set.

ihuRetrieveSampledData Function

This function returns sampled data based on the raw samples stored in archive. Interpolated, Trend, and Lab are examples of sampling modes.

Prototype

```
ihuRetrieveSampledData { in long serverhandle, in IHU_timestamp
```

```
StartTime,    in IHU_timestamp EndTime,    in ihuSamplingMode SamplingMode,  
in unsigned long NumberOfSamples,    in unsigned long IntervalMilliseconds,  
in/out IHU_RETRIEVED_DATA_RECORDS *DataRecords };
```

Remarks

Data can be requested using a number of samples or an interval. Set one to non zero and the other to zero. Each is a way to split the **duration = end - start** time into evenly spaced time intervals.

Returns

The function returns ihuSTATUS_OK when the values were successfully retrieved.

The function can return errors on read timeouts or when user is not a member of the ih Readers security group or if the tag was not found.

ihuRetrieveSampledDataEx Function

This function returns sampled data based on the raw samples stored in archive. Interpolated, Trend, and Lab are examples of sampling modes. This function is used with the hybrid sampling modes such as TrendToRaw or InterpolatedToRaw because it will return the sampling mode that was actually used.

Prototype

```
ihuRetrieveSampledDataEx { long serverhandle, // [in] which server to fetch  
from IHU_TIMESTAMP StartTime, // [in] Start time for the query IHU_TIMESTAMP  
EndTime, // [in] End time for the query ihuSamplingMode SamplingMode, // [in]  
The requested sampling mode unsigned long NumberOfSamples, // [in] 0 or num  
samples to return IHU_DATA_INTERVAL Interval, // [in] 0 or sampling interval  
in units provided by IntervalType IHU_RETRIEVED_DATA_RECORDS_EX *DataRecords  
// [in/out] - caller fills in tagnames of the structures and API will fill in  
the data };
```

Remarks

Data can be requested using a number of samples or an interval. Set one to non zero and the other to zero. To split the duration, divide the time from start to finish into evenly spaced time intervals.

If you are not using the Hybrid sampling modes you should use ihuRetrieveSampledData() function

Returns

The function returns ihuSTATUS_OK when the values were successfully retrieved.

The function can return errors on:

- Read timeouts.
- If the user is not a member of the iH Readers security group.
- If the tag was not found.

ihuRetrieveSampledDataEx2 Function

This function returns sampled data based on the raw samples stored in archive with filters and query modifiers using `pszFilterExpression` and `CriteriaString`. Interpolated, Trend, and Lab are examples of sampling modes. This function is typically used with the hybrid sampling modes such as `TrendToRaw` or `InterpolatedToRaw` because it will return the sampling mode that was actually used.

Prototype

```
ihuRetrieveSampledDataEx2 { long serverhandle, // [in] which server to fetch
from IHU_TIMESTAMP StartTime, //[in] Start time for the query IHU_TIMESTAMP
EndTime, //[in] End time for the query ihuSamplingMode SamplingMode,
//Sampling Modes MSO_Char *pszFilterExpression, // [in] Filter Expression
e.g. Tag > 20 IHU_FILTER_MODE FilterMode, // [in] Filter Modes MSO_Char
*CriteriaString, // [in] QueryModifiers to use with data reads unsigned long
NumberOfSamples, //[in] 0 or number of samples to return IHU_DATA_INTERVAL
Interval, // [in] 0 or sampling interval in units provided by IntervalType
IHU_RETRIEVED_DATA_RECORDS_EX *DataRecords //[in/out] - you fill in tagnames
of the structures and API will fill in the data };
```

Remarks

Data can be requested using a number of samples or an interval. Set one to non zero and the other to zero. To split the duration, divide the time from start to finish into evenly spaced time intervals.

If you do not want to do a filtering you can pass NULL value to the `pszFilterExpression` and `ihuExactTime` to the `FilterMode` or you can just use the `ihuRetrieveSampledData()` function.

Pass NULL value to the `CriteriaString` if you do not have a `QueryModifier`.

Returns

The function returns `ihuSTATUS_OK` when the values were successfully retrieved.

The function can return errors on:

- Read timeouts.
- If the user is not a member of the iH Readers security group.
- If the tag was not found.
- If the filter criteria or query modifier cannot be set.

ihuRetrieveCalculatedData Function

This function returns calculated data based on the raw samples stored in archive. Average, Minimum, and Count are examples of calculation modes.

Prototype

```
ihuRetrieveCalculatedData { in long serverhandle, in IHU_timestamp
StartTime, in IHU_timestamp EndTime, in ihuCalculationMode CalculationMode,
in unsigned long NumberOfSamples, in unsigned long IntervalMilliseconds,
in/out IHU_RETRIEVED_DATA_RECORDS *DataRecords };
```

Remarks

Data can be requested using a number of samples or an interval. Set one to non zero and the other to zero. To split the duration, divide the time from start to finish into evenly spaced time intervals.

Returns

The function returns `ihuSTATUS_OK` when the values were successfully retrieved.

The function can return errors on read timeouts or when user is not a member of the `ih Readers` security group or if the tag was not found.

ihuRetrieveCalculatedDataEx Function

This function returns calculated data based on the raw samples stored in archive. Average, Minimum, and Count are examples of calculation modes.

Prototype

```
ihuRetrieveCalculatedDataEx { long serverhandle, // [in] which server to  
fetch from IHU_TIMESTAMP StartTime, //[in] Start time of query IHU_TIMESTAMP  
EndTime, //[in] End time of query ihuCalculationMode CalculationMode, //[in]  
Calculation Mode unsigned long NumberOfSamples, //[in] Number of samples to  
be returned IHU_DATA_INTERVAL Interval, // [in] Interval in Milliseconds  
IHU_RETRIEVED_DATA_RECORDS_EX *DataRecords //[in/out] - you fill in tagnames  
of the structures and API will fill in the data };
```

Remarks

Data can be requested using a number of samples or an interval. Set one to non zero and the other to zero. To split the duration, divide the time from start to finish into evenly spaced time intervals.

Returns

The function returns `ihuSTATUS_OK` when the values were successfully retrieved.

The function can return errors on:

- Read timeouts.
- If the user is not a member of the `ih Readers` security group.
- If the tag was not found.

ihuRetrieveCalculatedDataEx3 Function

This function returns calculated data based on the raw samples stored in archive with filters and query modifiers using `pszFilterExpression` and `CriteriaString`. Average, Minimum, and Count are examples of calculation modes.

Prototype

```
ihuRetrieveCalculatedDataEx3 { long serverhandle, // [in] which server to  
fetch from IHU_TIMESTAMP StartTime, //[in] Start time of query IHU_TIMESTAMP
```

```

EndTime, //[in] End time of query ihuCalculationMode CalculationMode, //[in]
Calculation Mode MSO_Char *pszFilterExpression, // [in] Filter Expression
e.g. Tag > 20 IHU_FILTER_MODE FilterMode, // [in] Filter Modes MSO_Char
*CriteriaString, // [in] QueryModifiers to use with data reads ihuDataType
StateDataType, // DataType of the StateValue to compare for ihuStateCount
calculation mode ihuValue StateValue, // Value to compare for ihuStateCount
calculation mode unsigned long NumberOfSamples, IHU_DATA_INTERVAL Interval,
[in] Interval in Milliseconds IHU_RETRIEVED_DATA_RECORDS_EX *DataRecords
//[in/out] - you fill in tagnames of the structures and API will fill in the
data };

```

Remarks

Data can be requested using a number of samples or an interval. Set one to non zero and the other to zero. To split the duration, divide the time from start to finish into evenly spaced time intervals.

If you do not want to do a filtering, you can pass NULL value to the pszFilterExpression and ihuExactTime to the FilterMode or you can just use the ihuRetrieveCalculatedData() function.

Pass NULL value to the CriteriaString if you do not have a QueryModifier.

Returns

The function returns ihuSTATUS_OK when the values were successfully retrieved.

The function can return errors on:

- Read timeouts.
- If the user is not a member of the iH Readers security group.
- If the tag was not found.
- If the filter criteria or query modifier cannot be set.

Utility Functions

Utility Functions Overview

Utility functions are provided to ease programming. Time stamp conversion functions are necessary to produce the time stamps needed in most API functions.

Utility Functions

- IHU_timestamp_FromParts
- IHU_timestamp_ToParts

IHU_timestamp_FromParts Function

This function takes supplied time parts (date, hour, minutes, second, etc) and produces a UTC time stamp in the format needed by the Historian User API read and write calls.

Prototype

```
IHU_timestamp_FromParts { in int Year, in int Month, in int Day, in int Hour, in int Minute, in int Second, in long Subsecond, out IHU_timestamp *time stamp };
```

Remarks

The time parts passed in are assumed to be in the local time zone of the machine making the call.

The "Use Daylight Saving Time" setting of the machine making the call is used when converting to UTC

Returns

ihuSTATUS_OK on success or returns an error code.

IHU_timestamp_ToParts Function

This function takes the supplied UTC time stamp, and produces the following human-readable parts:

- Year
- Month
- Date
- Hour
- Minute
- Second
- Subsecond

This function can be used to convert the time stamps of retrieved data samples.

Prototype

```
IHU_timestamp_ToParts { in IHU_timestamp time stamp out int *Year, out int *Month, out int *Day, out int *Hour, out int *Minute, out int *Second, out long *Subsecond, };
```

Remarks

The time parts produced are in the local time zone of the machine making the call.

The "Use Daylight Saving Time" setting of the machine making the call is used when producing the time parts from UTC

Returns

ihuSTATUS_OK on success or returns an error code.

EnumeratedSet Functions

Enumerated Sets Functions Overview

You can use the Enumerated Set functions to create, modify, delete, and browse Enumerated sets and assign tags to those sets.

When querying data from tags using Enumerated sets, you will retrieve the string state name value by default. Use the `ihuEnumeratedSetRawValue` to indicate that you want to retrieve the numeric value.

When writing to a tag using Enumerated set, write the numeric value.

Use this function to retrieve all the enumerated sets that match the setname mask criteria. If you want to retrieve all sets then pass * as the mask.

EnumeratedSet Functions

- `ihuGetEnumeratedSets`
- `ihuEnumeratedSetAdd`
- `ihuEnumeratedSetRawValue`
- `ihuEnumeratedSetsFree`
- `ihuEnumeratedSetRename`
- `ihuEnumeratedSetDelete`
- `ihuEnumeratedStateAdd`
- `ihuEnumeratedStateModify`
- `ihuEnumeratedStateDelete`

ihuGetEnumeratedSets Function

Use the `ihuGetEnumeratedSets` function to retrieve all the enumerated sets that match the setname mask criteria. If you want to retrieve all sets then pass * as the mask.

Prototype

```
ihuParam IHUAPI ihuGetEnumeratedSets(
    in long serverhandle, in MSO Char *EnumeratedSetMask, out long
    *numberofrecords, out ihuEnumeratedSetRecordSet *EnumeratedSetRecordSet )
```

Remarks

The `ihuGetEnumeratedSets` function returns the following:

- Number of enumerated set records
- Enumerated sets

Returns

The `ihuGetEnumeratedSets` function returns `ihuSTATUS_OK` if the enumerated sets are retrieved successfully.

The `ihuGetEnumeratedSets` function returns errors if:

- There are any invalid input parameters.
- There are any out of memory issues while allocating dynamic memory.

ihuEnumeratedSetAdd Function

Use the `ihuEnumeratedSetAdd` function to add an enumerated set. The set passed to this function can have states in it or you can add states later.

Prototype

```
ihuEnumeratedSetAdd ( in long serverhandle, in ihuEnumeratedSetProperties  
*EnumeratedSet )
```

Remarks

If the set already exists it is overwritten by this new set.

Returns

The `ihuEnumeratedSetAdd` function returns `ihuSTATUS_OK` if the set is added successfully.

The `ihuEnumeratedSetAdd` function returns errors if:

- There are any invalid input parameters.
- There are any out of memory issues while allocating dynamic memory.
- The user is not a member of `ihTagAdmin` group. If the user sets the `Administrator` parameter to this security group, then this parameter will have the highest precedence.

ihuEnumeratedSetRawValue Function

Use the `ihuEnumeratedSetRawValue` function to indicate whether you want string or numeric values returned when reading tags using the enumerated set. By default, string values are retrieved if any tag is associated with an enumerated set. To retrieve numeric values, `SetRawValue` should be set to `TRUE`. If string values are not available, raw values are retrieved.

Prototype

```
ihuEnumeratedSetRawValue(  
in long serverhandle,  
in BOOL SetRawValue  
)
```

Remarks

The setting defaults to FALSE and only needs to be overridden if you want numeric values. The setting stays in place until the function is called again or when your program exits.

Returns

The `ihuEnumeratedSetRawValue` function returns `ihuSTATUS_OK` if the string or raw values are retrieved successfully.

The `ihuEnumeratedSetRawValue` function returns errors if:

- There are any invalid input parameters.
- The user is not a member of `ihTagAdmin` group. If the user sets the Administrator parameter to this security group, then this parameter will have the highest precedence.

ihuEnumeratedSetsFree Function

Use the `ihuEnumeratedSetsFree` function to clean up the enumerated set buffers created by API for `ihuGetEnumeratedSets()` API.

Prototype

```
ihuEnumeratedSetsFree(
    in long serverhandle,
    ihuEnumeratedSetRecordSet
    in *EnumeratedSetRecordSet)
```

Remarks

The memory is cleared from the enumerated set buffers.

Returns

The `ihuEnumeratedSetsFree` function returns `ihuSTATUS_OK` if the recordset is free.

The `ihuEnumeratedSetsFree` function returns errors if:

- There are any invalid input parameters.
- The user is not a member of `ihTagAdmin` group. If the user sets the `ihAdmin` parameter to this security group, then this parameter will have the highest precedence.

ihuEnumeratedSetRename Function

Use the `ihuEnumeratedSetRename` function to rename an enumerated set.

Prototype

```
ihuEnumeratedSetRename(
```

```
in long serverhandle, in MSO Char *oldEnumeratedSetName, in MSO Char  
*newEnumeratedSetName )
```

Remarks

You need to be an administrator of the set to be able to rename the set.

Returns

The ihuEnumeratedSetRename function returns ihuSTATUS_OK if the set is renamed successfully.

The ihuEnumeratedSetRename function returns errors if:

- There are any invalid input parameters.
- There are any out of memory issues while allocating dynamic memory.
- There is no set available with the old enumerated set name.
- The user is not a member of ihTagAdmin group. If the user sets the Administrator parameter to this security group, then this parameter will have the highest precedence.

ihuEnumeratedSetDelete Function

Use the ihuEnumeratedSetDelete function to delete an enumerated set.

Prototype

```
ihuEnumeratedSetDelete(  
in long serverhandle, in MSO Char *EnumeratedSetName )
```

Remarks

You need to be an administrator of the set to be able to delete a set.

Returns

The ihuEnumeratedSetDelete function returns ihuSTATUS_OK if the set is deleted successfully.

The ihuEnumeratedSetDelete function returns errors if:

- There are any invalid input parameters.
- There are any out of memory issues while allocating dynamic memory.
- There is no set available with the old enumerated set name.
- The user is not a member of ihTagAdmin group. If the user sets the ihAdmin parameter to this security group, then this parameter will have the highest precedence.

ihuEnumeratedStateAdd Function

Use the ihuEnumeratedStateAdd function to add a state to an existing enumerated state.

Prototype

```
ihuEnumeratedStateAdd( in long serverhandle, in MSO Char *EnumSetName, in
ihuEnumeratedSetState *EnumState )
```

Remarks

A state is added to the existing enumerated set. The data type of the new state should be the same as existing data type of other states.

If the state already exists it will be overwritten.

Returns

The ihuEnumeratedStateAdd function returns ihuSTATUS_OK if the state is added successfully.

The ihuEnumeratedStateAdd function returns errors if:

- There are any invalid input parameters.
- There are any out of memory issues while allocating dynamic memory.
- There is no set available with the old enumerated set name.
- The user is not a member of ihTagAdmin group. If the user sets the Administrator parameter to this security group, then this parameter will have the highest precedence.

ihuEnumeratedStateModify Function

Use the ihuEnumeratedStateModify function to modify an enumerated state.

Prototype

```
ihuEnumeratedStateModify( in long serverhandle, in MSO Char
*EnumeratedSetName, in MSO Char *EnumeratedStateName, ihuEnumeratedSetState
in *EnumStateToModify )
```

Remarks

A user cannot change the data type of the state. State names and their values can be changed. To change a name, pass the old name as a string and the new name inside the EnumStateToModify parameter.

Returns

The ihuEnumeratedStateModify function returns ihuSTATUS_OK if the state is modified successfully.

The ihuEnumeratedStateModify function returns errors if:

- There are any invalid input parameters.
- There is no set available with EnumeratedSetName.
- There is no set available with EnumeratedStateName.
- The user is not a member of ihTagAdmin group. If the user sets the ihAdmin parameter to this security group, then this parameter will have the highest precedence.

ihuEnumeratedStateDelete Function

Use the ihuEnumeratedStateDelete function to delete a state from a set.

Prototype

```
ihuEnumeratedStateDelete ( in long serverhandle, in MSO Char  
*EnumeratedSetName, in MSO Char *EnumeratedStateName  
)
```

Remarks

The enumerated state is deleted. The user cannot delete the state if only one state is available in the set. In this case, delete the entire set. You do not need to delete each state before deleting the set.

You need to be an administrator of the set to be able to delete states.

Returns

The ihuEnumeratedStateDelete function returns ihuSTATUS_OK if the state is deleted successfully.

The ihuEnumeratedStateDelete function returns errors if:

- There are any invalid input parameters.
- There is no set available with EnumeratedSetName.
- There is no set available with EnumeratedStateName.
- There are any out of memory issues while allocating dynamic memory.
- The user is not a member of ihTagAdmin group. If the user sets the Administrator parameter to this security group, then this parameter will have the highest precedence.

UserDefinedType Functions

UserDefined Type Overview

You can use the User Defined Type to create, modify, delete, and browse User Defined Data Types containing multifields. User defined data types needs to be added to the Data Archiver before they can be used in tags.

User Defined Type Functions

- ihuUserDefinedTypeAdd
- ihuUserDefinedTypeDelete
- ihuUserDefinedTypeRename
- ihuUserDefinedTypeExists
- ihuGetUserDefinedTypes
- ihuUserDefinedTypeSetProperties

- ihuUserDefinedTypeFreeProperties

ihuUserDefinedTypeAdd Function

Use the ihuUserDefinedTypeAdd function to create or add a MultiField type tag. The user defined type passed to this function can have multiple fields in it or you can add the fields later using the ihuUserDefinedTypeSetProperties().

You can also use the function to modify User Defined data types. If you add an User Defined data type that already exists with this name, the type is modified.

Prototype

```
ihuUserDefinedTypeAdd ( long serverhandle, // [in] connected server handle
ihuUserDefinedTypeProperties *InUserDefinedType //[in] UserDefined type that
needs to be added / modified
)
```

Remarks

If the set already exists it is overwritten by this new set. You need to be an ihTagAdmin to add or modify a set.

Returns

ihuSTATUS_INVALID_PARAMETER - If a NULL UserDefinedType or an invalid server handle is passed in.

ihuSTATUS_OK - On successful add.

ihuSTATUS_FAILED - If any error occurred.

ihuUserDefinedTypeDelete Function

Use the ihuUserDefinedTypeDelete function to delete a User Defined Type that you are no longer using.

Prototype

```
ihuUserDefinedTypeDelete ( long serverhandle, // [in] connected server handle
MSO_Char * UserDefinedTypeName // [in] UserDefined type name to be deleted
)
```

Remarks

You need to be an ihTagAdmin to be able to delete a User Defined type.

Returns

ihuSTATUS_INVALID_PARAMETER - If a NULL UserDefinedType or an invalid server handle is passed in.

ihuSTATUS_OK - On successful delete.

ihuSTATUS_FAILED - If any error occurred.

ihuUserDefinedTypeRename Function

Use the ihuUserDefinedTypeRename function to rename an enumerated set.

Prototype

```
ihuUserDefinedTypeRename  
  
(  
  
    long serverhandle, // [in] connected server handle  
  
    MSO_Char * UserDefinedTypeName, //[in] UserDefined type name which needs to  
    be renamed  
  
    MSO_Char *NewUserDefinedTypeName //[in] new UserDefined type name  
  
)
```

Remarks

You need to be an ihTagAdmin to be able to rename the User Defined Type.

Returns

ihuSTATUS_INVALID_PARAMETER - If a NULL UserDefinedType or an invalid server handle is passed in.

ihuSTATUS_OK - On successful rename.

ihuSTATUS_FAILED - If any error occurred.

ihuUserDefinedTypeExists Function

Use this function to check whether the user defined type exists.

Prototype

```
ihuUserDefinedTypeExists  
  
(  
  
    long serverhandle, // [in] connected server handle  
  
    MSO_Char * UserDefinedTypeName // [in] UserDefined type name  
  
)
```

Returns

ihuSTATUS_INVALID_PARAMETER - if a NULL UserDefinedType or an invalid server handle is

passed in.

ihuSTATUS_OK - If the type exists.

ihuSTATUS_FAILED - If the type does not exist or the server could not be reached.

ihuGetUserDefinedTypes Function

Use this function to return the list of User Defined Types and its values. You need to call the ihuUserDefinedTypeFreeProperties() function to release the memory allocations after calling the ihuGetUserDefinedTypes() function.

Prototype

```
ihuGetUserDefinedTypes
(
    long serverhandle, // [in] connected server handle

    MSO_Char *StructSetMask, //[in] Pass * to get the all the Structure or pass
    the name of the structure you want to retrieve.

    long *numberofrecords, //[out] Number of records

    ihuUserDefinedTypeRecordSet *RecordSet //[out] returns the records for the
    UserDefined defined types
)
```

Remarks

You need to be an ihReader to get the list. The list is returned in the Recordset parameter.

Returns

ihuSTATUS_INVALID_PARAMETER - If you pass NULL values for mask or recordset or number of records.

ihuSTATUS_OK - On successful completion.

ihuSTATUS_FAILED - If any error occurred.

ihuUserDefinedTypeSetProperties Function

This function is used to set the user defined type properties. You have to add the type using the ihuUserDefinedTypeAdd() call.

Prototype

```
ihuUserDefinedTypeSetProperties
(
    long serverhandle, // [in] connected server handle
```

```
ihuUserDefinedTypeProperties *UserDefinedType //[in] UserDefined type that  
needs to be added / created  
  
)
```

Remarks

You need to be an ihTagAdmin to set the properties.

Returns

ihuSTATUS_INVALID_PARAMETER - If an invalid serverhandle or NULL property pointer is passed in.

ihuSTATUS_OK - On successful set.

ihuSTATUS_FAILED - If any error occurred.

ihuUserDefinedTypeFreeProperties Function

This function is used to free the memory allocated by the ihuGetUserDefinedTypes() call.

Prototype

```
ihuUserDefinedTypeFreeProperties  
  
(  
  
    ihuUserDefinedTypeRecordSet *UserDefinedTypeRecordSet //[in]  
    UserDefinedTypeRecordSet that needs to be freed.  
  
)
```

Remarks

You need to call this once to free all memory in the recordset returned by ihuGetUserDefinedTypes(). Do not free the memory in your program.

Returns

Void

Publish Functions

Publish Functions Overview

The Publish functions present a unified interface for managing data forwarding of local Historian server tags to one or more remote destination servers via the Server-to-Server Distributors. Publishing a tag initiates continuous transmission of its historical data values to a destination server.

Publish Functions

The following are the Publish functions:

- ihuPublishAddTag
- ihuPublishRemoveTag
- ihuPublishTagCloseCache
- ihuPublishTagGetNumericPropertyByTagname
- ihuPublishTagGetNumericPropertyByIndex
- ihuPublishTagGetNumericPropertyByIndex
- ihuPublishTagGetStringPropertyByTagname
- ihuPublishTagGetStringPropertyByIndex
- ihuPublishSetTagProperties
- ihuPublishTagSetNumericProperty
- ihuPublishTagSetStringProperty
- ihuPublishTagClearProperties
- ihuPublishGetDestinationServer
- ihuPublishSetDestinationServer

ihuPublishAddTag Function

Use the ihuPublishAddTag function to publish a tag to a destination server. This creates a tag in the destination server.

Prototype

```
ihuPublishAddTag( long hSourceServer, MSO Char *InterfaceName, MSO Char
*SourceTagname, );
```

Remarks

Once published, any new data for the SourceTagname will be written to the DestinationTagname.

The tag will be added to the destination with some default parameters. You can change them after the tag is added via the ihuPublishGetTagPropertiesToCache and ihuPublishSetTagProperties calls.

Returns

The ihuPublishAddTag function returns the ihuSTATUS_OK if the published tag has been added.

The `ihuPublishAddTag` function returns an error if the tag could not be added to the destination server.

ihuPublishRemoveTag Function

Use the `ihuPublishRemoveTag` function to stop a tag from being published via the specified collector.

Prototype

```
ihuPublishRemoveTag ( long hServer, MSO Char *DestinationTagname, MSO Char *SourceTagname);
```

Remarks

This call does not affect the tag in the source server but marks it as deleted in the destination server. If you only want to stop data collection, then you need to disable collection.

Returns

The `ihuPublishRemoveTag` function returns the `ihuSTATUS_OK` if the published tag has been removed.

ihuPublishTagCloseCache Function

Use the `ihuPublishTagCloseCache` function to close a cache returned by the `ihuPublishGetTagPropertiesToCache`.

Prototype

```
ihuPublishTagCloseCache  
(void);
```

Remarks

You should close cache to avoid memory leaks.

Returns

The `ihuPublishTagCloseCache` function returns `ihuSTATUS_OK` if the cache has been closed.

ihuPublishGetTagPropertiesToCache Function

Use the `ihuPublishGetTagPropertiesToCache` function to retrieve a list of tags published via the named Server-to-Server Distributor. Match tags with the exact tag name or a name with a specified wildcard mask.

A separate published tag properties cache exists per process thread. To avoid confusion and unexpected results, ensure that all cache related function calls are invoked on the same thread. Mixing threads will reference two or more distinct caches which enable concurrent access to multiple, different cached results at the cost of additional memory.

Prototype

```
ihuPublishGetTagPropertiesToCache
(
    long hServer, MSO Char *DistributorName, MSO Char *DestinationTagnameOrMask,
    out int *NumTagsFound );
```

Remarks

The ihuPublishGetTagPropertiesToCache function returns the number of tags that have been found in Numtags.

Use this function to retrieve tags in preparation of getting and setting string or numeric tag properties via the ihuPublishTagGetNumericPropertyByTagname or ihuPublishTagSetStringProperty and similar functions.

Returns

The ihuPublishGetTagPropertiesToCache function returns the ihuSTATUS_OK if the call is completed successfully, even if no tags are found.

The ihuPublishGetTagPropertiesToCache function returns errors if there are any out of memory issues while allocating dynamic memory.

ihuPublishTagGetNumericPropertyByTagname Function

Use the ihuPublishTagGetNumericPropertyByTagname function to retrieve a numeric tag property such as deadband for a tag. The tag is identified by name within the cache previously populated by ihuPublishGetTagPropertiesToCache.

Prototype

```
ihuPublishTagGetNumericPropertyByTagname
(
    MSO Char *Tagname, ihuTagProperties TagProperty, *Value );
```

Remarks

The ihuPublishTagGetNumericPropertyByTagname function returns the numeric tag property value.

Returns

The ihuPublishTagGetNumericPropertyByTagname function returns the ihuSTATUS_OK if the numeric tag property value is found.

The ihuPublishTagGetNumericPropertyByTagname function returns an error if there are any out of memory issues while allocating dynamic memory.

ihuPublishTagGetNumericPropertyByIndex Function

Use the ihuPublishTagGetNumericPropertyByIndex function to retrieve an individual, numeric tag property from the cache.

Prototype

```
ihuPublishTagGetNumericPropertyByIndex  
( int Index, ihuTagProperties TagProperty, double *Value);
```

Remarks

The ihuPublishTagGetNumericPropertyByIndex function returns the numeric tag property retrieved. The index is zero based.

Returns

The ihuPublishTagGetNumericPropertyByIndex function returns the ihuSTATUS_OK if the numeric tag property from cache is retrieved on a zero based index.

The ihuPublishTagGetNumericPropertyByIndex function returns an error if there are any out of memory issues while allocating dynamic memory.

ihuPublishTagGetStringPropertyByTagname Function

Use the ihuPublishTagGetStringPropertyByTagname function to retrieve an individual string tag property by tagname from cache.

Prototype

```
ihuPublishTagGetStringPropertyByTagname  
( MSO Char *Tagname, ihuTagProperties TagProperty, MSO Char *Value, int  
ValueLength );
```

Remarks

The ihuPublishTagGetStringPropertyByTagname function returns the individual string tag property by tagname from cache.

Returns

The ihuPublishTagGetStringPropertyByTagname function returns ihuSTATUS_OK if the individual string tag property is retrieved by tagname from cache.

The ihuPublishTagGetStringPropertyByTagname function returns an error if there are any out of memory issues while allocating dynamic memory.

ihuPublishTagGetStringPropertyByIndex Function

Use the ihuPublishTagGetStringPropertyByIndex function to retrieve an individual, string tag property from cache. The index is zero based.

Prototype

```
ihuPublishTagGetStringPropertyByIndex( int Index,
```

```
ihuTagProperties TagProperty, MSO Char *Value, int ValueLength );
```

Remarks

The ihuPublishTagGetStringPropertyByIndex function returns the string tag property from cache. The index is zero based.

Returns

The ihuPublishTagGetStringPropertyByIndex function returns ihuSTATUS_OK if the string tag property is retrieved by zero based index from the cache.

The ihuPublishTagGetStringPropertyByIndex function returns an error if there are any out of memory issues while allocating dynamic memory.

ihuPublishSetTagProperties Function

Use the ihuPublishSetTagProperties function to define properties such as deadband for a published tag. Set the properties via ihuPublishTagSetStringProperty or ihuPublishTagSetNumericProperty before calling this function.

A separate collection of tag properties is available for each process thread, similar to the published tag properties cache. To avoid mix-up ensure that the ihuPublishSetTagProperties, ihuPublishTagSetNumericProperty, ihuPublishTagSetStringProperty and ihuPublishTagClearProperties are invoked on the same thread.

Prototype

```
ihuPublishSetTagProperties( long hServer, MSO Char *InterfaceName,  
MSO Char *DestinationTagname, );
```

Remarks

Use the ihuPublishSetTagProperties function to set tag properties after the tag has been added using the ihuPublishAddTag function.

Returns

The ihuPublishSetTagProperties function returns ihuSTATUS_OK if the properties have been defined for the published tags.

The ihuPublishSetTagProperties function returns an error if there are any out of memory issues while allocating dynamic memory.

ihuPublishTagSetNumericProperty Function

Use the ihuPublishTagSetNumericProperty function to define a numeric tag property before invoking the ihuPublishSetTagProperties.

Prototype

```
ihuPublishTagSetNumericProperty  
(  
    ihuTagProperties TagProperty, double Value );
```

Remarks

The ihuPublishTagSetNumericProperty function returns the numeric tag property.

Returns

The ihuPublishTagSetNumericProperty function returns ihuSTATUS_OK if the numeric tag property is set.

The ihuPublishTagSetNumericProperty function returns errors if:

- There are any invalid input parameters.
- There are any out of memory issues while allocating dynamic memory.

ihuPublishTagSetStringProperty Function

Use the ihuPublishTagSetStringProperty function to define a string property of the tag before invoking the ihuPublishSetTagProperties.

Prototype

```
ihuPublishTagSetStringProperty(  
    ihuTagProperties TagProperty, MSO Char *Value );
```

Remarks

The ihuPublishTagSetStringProperty function returns the string property of the tag.

Returns

The ihuPublishTagSetStringProperty function returns ihuSTATUS_OK if the string property of the tag is set.

The ihuPublishTagSetStringProperty function returns errors if:

- There are any invalid input parameters.
- There are any out of memory issues while allocating dynamic memory.

ihuPublishTagClearProperties Function

Use the ihuPublishTagClearProperties function to clear any previously established tag properties.

Prototype

```
ihuPublishTagClearProperties
( void );
```

Remarks

The ihuPublishTagClearProperties function clears the previously established tag properties.

Returns

The ihuPublishTagClearProperties function returns ihuSTATUS_OK if the previously established tag properties are cleared.

The ihuPublishTagClearProperties function returns an error if there are any out of memory issues while allocating dynamic memory.

ihuPublishGetDestinationServer Function

Use the ihuPublishGetDestinationServer function to retrieve the name of the destination Proficy Historian server that is configured for a distributor.

Prototype

```
ihuPublishGetDestinationServer(
long hServer, MSO Char *InterfaceName, MSO Char **DestinationServer );
```

Remarks

The ihuPublishGetDestinationServer function returns the destination Historian server name.

Returns

The ihuPublishGetDestinationServer function returns ihuSTATUS_OK if the destination Historian server name is retrieved.

The ihuPublishGetDestinationServer function returns an error if there are any out of memory issues while allocating dynamic memory.

ihuPublishSetDestinationServer Function

Use the ihuPublishSetDestinationServer function to associate the given destination Proficy Historian server to a collector.

Prototype

```
ihuPublishSetDestinationServer(
long hServer,
```

```
MSO Char *InterfaceName,  
MSO Char *DestinationServer  
);
```

Remarks

The ihuPublishSetDestinationServer function returns the Historian server name that is associated with the collector. The new name will be assigned to the collector without restarting it and can be used at any time the distributor is restarted.

Returns

The ihuPublishSetDestinationServer function returns ihuSTATUS_OK if the Historian server name that is associated with the collector is retrieved.

The ihuPublishSetDestinationServer function returns an error if there are any out of memory issues while allocating dynamic memory.

Error Codes

Historian User API Error Codes

The following error codes may be encountered while using the Historian User API:

100	ihuSTATUS_FAILED	Generic failure.
101	ihuSTATUS_API_TIMEOUT	Server machine name not found, or server found but archiver service not running.
102	ihuSTATUS_NOT_CONNECTED	Not currently connected to an Historian server.
103	ihuSTATUS_INTERFACE_NOT_FOUND	Interface not found.
104	ihuSTATUS_NOT_SUPPORTED	reserved

105	ihuSTATUS_DUPLICATE_DATA	ihuWriteData was called with error_on_replace = TRUE and the supplied data would have overwritten existing data.
106	ihuSTATUS_DUPLICATE_DATA	Server found, but invalid username or password.
107	ihuSTATUS_ACCESS_DENIED	Access is denied by the Historian server. Check username/password or security group membership.
108	ihuSTATUS_WRITE_IN_FUTURE	Write time stamp is too far in the future.
109	ihuSTATUS_WRITE_ARCH_OFFLINE	There is no archiver to hold the write time stamp.
110	ihuSTATUS_ARCH_READONLY	The archive to hold the write time stamp is marked as read-only.
111	ihuSTATUS_WRITE_OUTSIDE_ACTIVE	The write time stamp is before the active hours (now - "data is read only after") setting.

112	ihuSTATUS_WRITE_NO_ARCH_AVAILABLE	No archive is available to hold the write time stamp.
113	ihuSTATUS_INVALID_TAGNAME	Tagname used is not valid. Tagname does not exist in the Historian server.
114	ihuSTATUS_LIC_TOO_MANY_TAGS	Exceeded tag license count on the server.
115	ihuSTATUS_LIC_TOO_MANY_USERS	Exceeded user license count on the server.
116	ihuSTATUS_LIC_INVALID_LIC_DLL	An invalid license DLL is installed.
117	ihuSTATUS_NO_VALUE	No value has been passed to the function.
118	ihuSTATUS_NOT_LICENSED	Your installation of Historian is not licensed.
119	ihuSTATUS_CALC_CIRC_REFERENCE	reserved
120	ihuSTATUS_DUPLICATE_INTERFACE	reserved
121	ihuSTATUS_BACKUP_EXCEEDED_SPACE	reserved

122	ihuSTATUS_INVALID_SERVER_VERSION	You are attempting to use this API on an invalid version of Historian.
123	ihuSTATUS_DATA_RETRIEVAL_COUNT_EXCEEDED	You tried to request too many samples in one read request.
124	ihuSTATUS_INVALID_PARAMETER	Generic failure when an invalid value is passed into the user API.

Sample Programs

Sample Programs Overview

Sample programs are provided with the API, demonstrating how to perform common tasks. The following sample programs are supplied with the User API:

- CollectorLike.c - shows an example of a program that writes thousands of samples per second to Historian tags.
- CollectorLikeSAF.c- shows an example of callbacks and store and forward in the user API.
- PlotLike.cpp - shows an example of a program that retrieves calculated and interpolated data suitable for plotting.
- ReportLike.cpp - shows how to retrieve raw data samples from Historian.
- MigrationLike.c - shows how to transfer legacy historical data into Historian.
- QueryModifiers.c - shows how to retrieve data using Query Modifiers.
- Blobdatatype.c &endash; shows how to create tags of blob (binary object) data type and read and write data.
- ByTagId &endash; shows how to read and write data for a tag by using its tagid instead of tagname.

No guidance is given regarding application conversion from other products or APIs, including converting from the Historian SDK.

The reader is assumed to be familiar with the Historian features and functionality. Refer to the Historian Electronic Book for product information. This is especially important if you are using

security groups with Historian, since the applications calling into the User API are limited by the security access granted at the server.

About compiling the samples

Only a release build configuration is provided. A debug configuration is not provided because the release mode configuration does produce a program database and has compiler optimizations turned off, you can easily step through the sample code in the debugger.

Review the sample program source code for additional details.

NOTE: The sample programs provided in this help document are for review only. Compilable source code is included in the Historian User API installation directory.

CollectorLike

This sample demonstrates reading string and numeric tag properties and writing groups of data samples to multiple tags. Data is sent to the archiver until a key is pressed. Sample code is given for writing data of different data types and with different data qualities. Tips are included for achieving optimal performance. Demonstrates iterating through all archiver tags looking for tags belonging to the collector.

PlotLike Sample Program

This sample program demonstrates reading numeric tag properties and reading evenly spaced sampled or calculated data suitable for plotting. Percent good data quality indicators are described. Any comments retrieved are printed to the program window. Finally, an example is given for how to write a comment in case your trending application allows users to enter comments.

ReportLike Sample Program

This sample program illustrates how raw samples are retrieved, one tag per call, by timespan and by number of samples. Comments are retrieved if they exist. The retrieved samples could be used in custom quality calculations or to implement calculation modes not available in the Historian. An example of calculating a raw maximum is given. The program has a loop that fetches the current value of a tag until a key is pressed.

MigrationLike Sample Program

This sample program shows how you can use the Historian User API to migrate data from a legacy system into Historian. Tips are given for most efficient order to send data to minimize archive disk space and minimize migration time. Also, the sample program demonstrates how to add tags into the Historian Server.